Assignment 5: Clustering and Initialization

Daniyar Ulkhan
Statistical Methods for Machine Learning, A.Y. 2025/26

Abstract

This project compares random initialization and k-means++, two ways of choosing starting centres for k-means. Both use the same clustering loop and were written using NumPy. The experiments use 900 separated synthetic points and 2,000 MNIST digit images. The extensions make the synthetic groups overlap and add noise features which comes up to 784 dimensions. After we run each setting 20 times per method resulting in 320 runs, where both methods reach the same final inertia on the separated data. Random initialization has a slightly lower average inertia and standard deviation on MNIST and the overlapping groups. K-means++ has the lower values in the synthetic 784-dimensional setting. Neither method performs better in every setting.

1 Question and scope

K-means assigns observations to their nearest centre, which then moves each centre to the average of the group, after it repeats these steps until the groups stop changing. The starting centres at the end will affect the result. And this project asks whether choosing them differently changes the final inertia and makes the results more consistent between runs.

The first method chooses its starting centres uniformly at random from the data. The second uses k-means++ which was proposed by Arthur and Vassilvitskii [1], which gives more weight to observations farther from centres already chosen. After this step both methods use the same Lloyd update loop. The starting rule is the only difference between them.

Following the assignment objectives we use synthetic data with clear groups and a real dataset with several classes and mutliple features. Then we compare inertia and its variation over repeated runs, where for MNIST - the algorithm groups images by their pixel values without using the digit labels. The two extensions examine overlapping groups and extra noise features.

2 Datasets and evaluation protocol

2.1 Synthetic data

We generate 900 points in two dimensions: 300 around each of the centres $(-5, 0)$, $(0, 0)$ and $(5, 0)$. Both coordinates use independent Gaussian noise with standard deviation 0.7. Seed 42 makes the generated points repeatable.

The result is three separated clouds, which are shown on the left of Figure 1. We choose $k = 3$ because the data were generated from three groups. Neither method receives the generating centres, both must choose their own starting centres from the 900 points.

2.2 MNIST

Given MNIST dataset contains images of handwritten digits from 0 to 9. Its training archive has 60,000 images of size 28×28 pixels [2]. We then select 2,000 images at random without repeats using seed 42 and ofcourse use this same sample in every run.

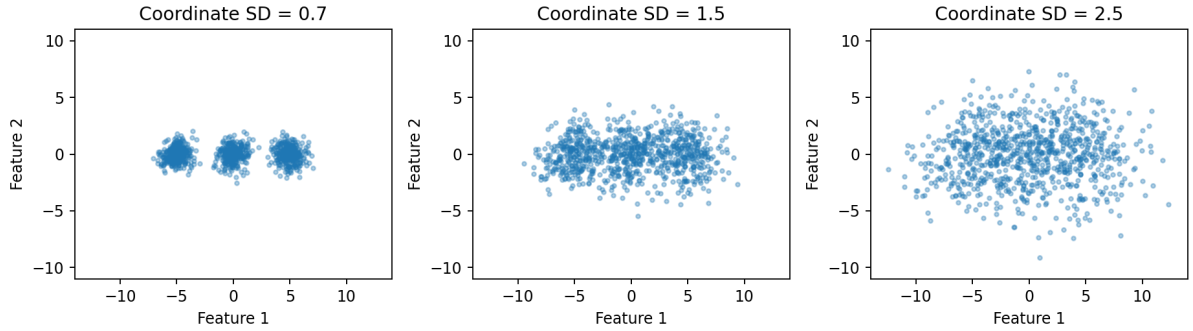


Figure 1: The synthetic family at coordinate standard deviations 0.7, 1.5 and 2.5, using the same centres and Gaussian offsets. Here the left panel is the original separated dataset and the other panels show increasing overlap. Axes are shared, and the generating centres are not supplied to the algorithms.

As a result each image becomes a row of 784 pixel values. We divide the values by 255 to put them in $[0, 1]$. This gives us a dataset with many features, while the smaller sample keeps the repeated runs manageable.

We set $k = 10$ since our MNIST dataset contains ten digit categories. Also a cluster is not automatically a digit category: different handwriting styles of the same digit may fall into separate groups, and similar-looking images of different digits may be grouped together to which we have to account.

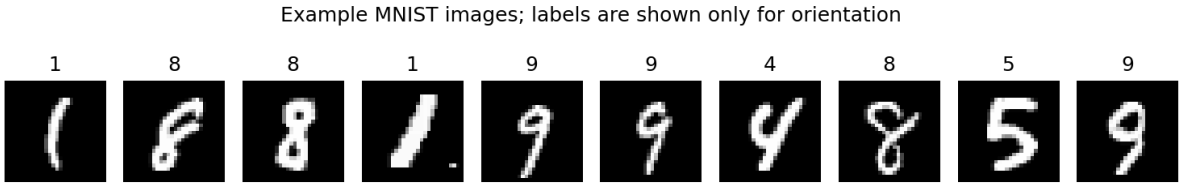


Figure 2: The first ten images in the fixed MNIST sample. Labels identify the displayed examples, but are not used by the clustering algorithm.

The notebook downloads the archive from the location used by Keras and checks its SHA-256 checksum against the official loader source [3]. It reads the archive directly, so Keras and TensorFlow are not needed.

2.3 Overlapping clusters

For the first extension, we keep the three generating centres, 900 points and $k = 3$. We increase the standard deviation from 0.7 to 1.5 and 2.5. The same Gaussian random values generated with seed 42 are scaled to each spread and added to the centres. This makes the groups wider without drawing new random values for every setting. The groups then overlap more, as Figure 1 shows.

The 0.7 setting reuses the original synthetic data and its results, and only the other two settings need extra runs. The clustering code is not given the original group labels. Since when we change the spread it also changes the scale of inertia, so we compare the methods at each spread separately.

2.4 Additional dimensions

The second extension keeps the original two coordinates and adds independent Gaussian noise features with mean zero and standard deviation 0.7. The total feature counts are 2, 10, 50, 200 and 784. Every setting has 900 points and $k = 3$.

We generate the extra features once with seed 43. At each higher feature count, we keep the earlier columns and add more from the same noise matrix. The extra features are generated in the same way for all three groups. This tests what happens when irrelevant information is added to the original data. MNIST is a separate real-data experiment with 784 features.

The two-dimensional results are reused from the original experiment. All spreads, feature counts and run seeds were chosen before the experiments were run.

2.5 Why the data are kept fixed

We want to measure the effect of the starting centres and for this reason, every run at a given setting uses the same observations, features and value of k. We prepared the synthetic data and MNIST sample only once before the repeated runs.

Afterwards we calculate inertia on the observations being clustered, without a separate training/test split. This lets us compare starting rules on the same data. The MNIST test archive is unused, so these results don't measure performance on new images.

3 Implementation

3.1 Objective and distances

For observations $x_1, \ldots, x_n$ and centres $c_1, \ldots, c_k$, the objective is

$$J(C) = \sum_{i=1}^{n} \min_{1 \le j \le k} \|x_i - c_j\|^2. \tag{1}$$

This sum is called inertia. With the same data and k, so in our case lower inertia means that observations are closer to their group centres.

For MNIST, distance is calculated from the 784 pixel values. Inertia measures how tightly the images are grouped. Its size also depends on the number of observations, number of features and their scale. We therefore compare the methods within each setting.

The function `squared_distances` calculates the squared distance from every observation to every centre using

$$\|x - c\|^2 = \|x\|^2 + \|c\|^2 - 2x^{\mathsf{T}}c.$$

This formula uses less memory than storing all coordinate differences at once. Any small negative distances caused by rounding are set to zero.

3.2 Starting centres

Random initialization chooses k different observations, with each observation equally likely to be selected.

K-means++ chooses the first centre uniformly at random. For each observation, D_i^2 is its squared distance to the nearest centre already chosen. The probability of choosing it as the next centre

is

$$p_i = \frac{D_i^2}{\sum_{r=1}^{n} D_r^2}. \tag{2}$$

The distances are updated after each new centre, until k centres are chosen. Observations farther from the existing centres have a greater chance of being selected. The method still makes a random choice and it doesnt always choose the farthest point.

Furthermore observation cannot be selected twice: If all distances are zero, the code chooses another unused observation uniformly at random. This avoids division by zero when the data has duplicate points.

3.3 Shared update loop

The function `fit_kmeans` uses the following procedure after either initialization:

1. Assign every observation to its nearest centre.
2. Replace each non-empty cluster centre with the coordinate-wise mean of its assigned observations.
3. Reassign the observations and record the new inertia.
4. Repeat until assignments no longer change, or 200 updates have been completed.

If a cluster is empty the code starts to moves its centre to an observation farthest from its assigned centre. diffrent observations are used if several clusters are empty for these replacements. When distances tie we set the NumPy to consistently choose the first minimum.

Each run returns the centres, assignments, initial and final inertia, inertia history, update count and whether it converged. Both starting methods use the same code. The clustering steps are written using NumPy rather than a ready-made clustering function.

These steps lower inertia or leave it unchanged, apart from rounding error and also our algorithm can still stop when a better clustering exists. As a result 2 runs can converge and finish with different inertia values.

4 Experimental design

Table 1 lists the eight settings. Each has 20 runs per method, using seeds 0 through 19, giving $8 \times 2 \times 20 = 320$ runs. The separated two-dimensional results are shared by the original experiment and both extensions. We run each method once per seed and then save every result.

Table 1: Experimental settings. Each condition uses 20 trials per method and a 200-update cap.

Conditions	Observations	Features	k	Fits
Separated synthetic	900	2	3	40
MNIST	2,000	784	10	40
Overlap: $\sigma = 1.5, 2.5$	900	2	3	80
Added features: $d = 10, 50, 200, 784$	900	10–784	3	160

The seeds allow the runs to be repeated because the same seed number doesnt give the methods same starting centres, since they choose those centres differently.

We report the mean final inertia, sample standard deviation, minimum and maximum from the 20 runs. The standard deviation uses denominator $20-1$. Relative standard deviation is $100s/\bar{J}$, where s is the sample standard deviation and $\bar{J}$ is mean inertia. It describes the variation as a percentage of the mean.

Here, stability means how much final inertia changes between runs on the same data. We consider it with our mean since consistently reaching a poor score would still give low variation. The code checks that every run converges and that inertia does not increase beyond rounding tolerance. All 320 runs passed and the largest update count was 83, which is below our limit of 200.

5 Results

Figure 3 shows the final inertia values, where we can see that each dot is one run. The box covers the middle half of the results, and the line inside marks the median. Small horizontal offsets keep overlapping dots visible. Lower inertia means more compact clusters on the same dataset, while a smaller vertical spread means less variation between runs. We compare the methods within each dataset, since the raw inertia values from different datasets camt be directly compared.

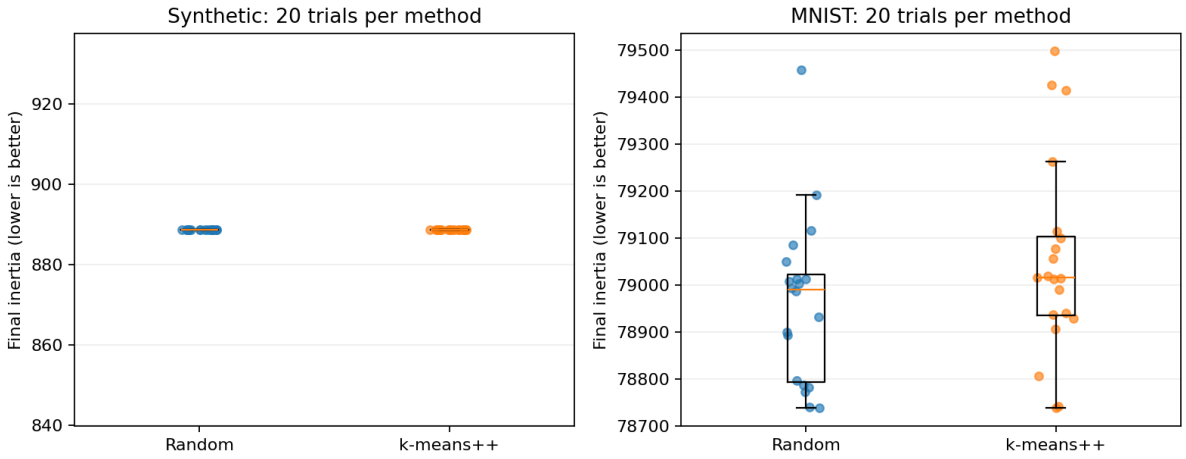


Figure 3: Final inertia over 20 runs per method. The synthetic outcomes are similiar, while the MNIST outcomes vary across seeds. Lower values indicate more compact clusters within a panel.

5.1 Synthetic data

Both methods reached final inertia 888.711 in every run, to the displayed precision (Table 2). The standard deviation is zero, and all 40 runs converged.

The groups are clearly separated and changing the starting centres didn't change final inertia in the runs. As presented neither method had an advantage on this dataset. This agreement doesnt prove that the result is the best possible clustering.

Table 2: Final inertia on the synthetic data, 20 trials per method.

Initialization	Mean	Sample SD	Minimum	Maximum
Random	888.711	0.000	888.711	888.711
K-means++	888.711	0.000	888.711	888.711

5.2 MNIST

The MNIST results vary across seeds (Table 3). Random initialization has mean final inertia 78,962.916, compared with 79,049.737 for k-means++. The difference is 86.821, or approximately 0.11% of the k-means++ mean.

Random initialization also varied less between runs: its sample standard deviation is 176.468, compared with 211.037 for k-means++. It gave slightly tighter groups on average in this experiment. All 40 MNIST runs converged.

Table 3: Final inertia on the fixed MNIST subset, 20 trials per method.

Initialization	Mean	Sample SD	Minimum	Maximum
Random	78,962.916	176.468	78,737.683	79,457.876
K-means++	79,049.737	211.037	78,737.655	79,497.417

The best single scores are almost equal: 78,737.683 for random initialization and 78,737.655 for k-means++. Although k-means++ has the slightly better best run, random initialization has the lower average. This is why the comparison includes all runs.

5.3 Overlapping clusters

The overlapping groups give small differences between runs (Table 4). At $\sigma = 1.5$, random initialization has mean inertia 3,730.362, compared with 3,730.373 for k-means++. At $\sigma = 2.5$, the means are 8,665.846 and 8,666.099. Both differences are below 0.003% of the k-means++ mean. Random initialization also has slightly lower standard deviations.

The relative standard deviations increase from zero at $\sigma = 0.7$ to about 0.0061% and 0.0065% at 1.5, then 0.0157% and 0.0175% at 2.5, for random initialization and k-means++ respectively. Starting centres therefore make some difference on the overlapping data, but the variation is small. Higher inertia at a wider spread alone does not show that initialization worked worse.

Table 4: Overlap results: mean and sample standard deviation of final inertia. R denotes random initialization; ++ denotes k-means++. Each row uses 20 trials per method.

σ	Mean R	Mean ++	SD R	SD ++
0.7	888.711	888.711	0.000	0.000
1.5	3,730.362	3,730.373	0.229	0.243
2.5	8,665.846	8,666.099	1.362	1.517

5.4 Additional irrelevant dimensions

With 2, 10, 50 and 200 features, all runs give the same final inertia at each feature count (Table 5). At 784 features the k-means++ has mean inertia 344,944.397, compared with 345,294.461 for random initialization whcih are about 0.101% lower. It also varies less between runs where standard deviation 804.252 versus 1,294.718. The relative standard deviations are 0.2332% and 0.3750%, respectively.

Both methods share the best observed inertia, 344,764.151. Five of the 20 random runs and six of the 20 k-means++ runs reach this value within an absolute tolerance of 10^{-7}. The other scores produce the difference in mean and spread shown in Table 5. K-means++ performed better in this setting, but also other high-dimensional datasets could give different results.

Table 5: Added-feature experiment: mean and sample standard deviation of final inertia. R denotes random initialization; ++ denotes k-means++. Raw inertia is compared within each feature count.

d	Mean R	Mean ++	SD R	SD ++
2	888.711	888.711	0.000	0.000
10	4,338.609	4,338.609	0.000	0.000
50	21,990.794	21,990.794	0.000	0.000
200	88,184.207	88,184.207	0.000	0.000
784	345,294.461	344,944.397	1,294.718	804.252

6 Discussion and limitations

As a result the difference between methods is minor where both gave the same final inertia on the separated data and after adding features up to 200 dimensions. Random initialization had slightly lower averages on MNIST and the overlapping groups. With 784 synthetic features, k-means++ gave a lower mean and standard deviation. The starting rule therefore mattered more in some experiments than in others: this may be because some datasets have clearer groups than others which leads to similiar results, on opposite when groups overlap there could be many irrelevent features which would make the grouping mudy, and the choice of starting centres has more influence.

Arthur and Vassilvitskii give a bound on the average inertia of k-means++ relative to the best possible clustering [1]. This does not promise that it will beat random initialization on every dataset or set of runs. Our mixed results don't contradict that bound.

There are limits to these comparisons. Increasing the spread changes both overlap and the variation inside each group. The extra dimensions do contain noise and we could explore adding useful features that couldve had a different effect. Each setting uses one generated dataset or image sample, fixed k and 20 run seeds. Other samples or cluster counts may give different results

For MNIST, a small change in writing style or stroke position can change pixel distances even when the digit stays the same. Inertia therefore measures how compact the image groups are,so its not a digit-recognition accuracy. We also didn't measure performance on newer observations or running time which could be explored in later iterations. Equal final inertia doesnt mean that all observations have the same assignments.

7 Conclusion

Changing the starting centres affected some results more than others. Both methods gave equal final inertia on the separated data and with added features through 200 dimensions. Random initialization had a slightly lower mean inertia and less variation on MNIST and the overlapping groups. K-means++ had the lower mean and variation in our 784-dimensional synthetic experiment, even though both methods reached the same best score.

References

[1] D. Arthur and S. Vassilvitskii (2007). *k-means++: The Advantages of Careful Seeding.* Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 1027–1035. Publication record. Paper copy.

[2] Y. LeCun, C. Cortes and C. J. C. Burges. *MNIST handwritten digit dataset*. Dataset description and archive documentation: `https://keras.io/api/datasets/mnist/`. The documented dataset licence is Creative Commons Attribution-ShareAlike 3.0.

[3] Keras contributors. *MNIST dataset loader source*, including the archive URL and checksum. `https://github.com/keras-team/keras/blob/master/keras/src/datasets/mnist.py`.

[4] N. Cesa-Bianchi. *Statistical Methods for Machine Learning: Experimental Projects*, A.Y. 2025/26, Assignment 5 and general project specifications. Course project instructions.

Required declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.